PROJECT REPORT

PYTHON QUIZ GAME

A Command-Line Based Quiz Application Using Python

Project Type	Python Programming Project
Application	Interactive Quiz Game
Platform	Command Line / Terminal
Technology	Python 3.8 or Later
Dependencies	Python Standard Library

Submitted as a structured project report along with the project repository.

ABSTRACT

The Python Quiz Game is a command-line application designed to provide an interactive question-and-answer experience using the Python programming language. The application presents multiple-choice questions, receives answers from the user, validates the responses, maintains a running score, and displays the final result after the quiz is completed. The project demonstrates fundamental programming concepts such as variables, lists and dictionaries, loops, conditional statements, functions, user input, and basic input validation. The application is intentionally simple so that its question bank and features can be extended easily.

Keywords: Python, Quiz Game, Command Line Application, Multiple Choice, Programming Project

1. INTRODUCTION

Interactive applications are useful for demonstrating programming concepts because they combine input, processing, decision-making, and output in one workflow. The Python Quiz Game applies these concepts to a familiar quiz format. A user is shown a series of questions with answer choices and selects an answer through the terminal. The program compares the response with the stored correct answer and updates the score accordingly.

The project is suitable as an introductory Python application because it can be implemented without external packages. Its simple architecture also makes it suitable for future improvements such as categories, difficulty levels, randomized questions, timers, high scores, and a graphical user interface.

2. PROBLEM STATEMENT

Create a Python-based quiz application that can present multiple-choice questions to a user, accept and validate the user's responses, determine whether each response is correct, calculate the total score, and clearly display the final result. The project should be simple to configure and should include clear documentation describing how to set up and run it.

3. OBJECTIVES

- Develop an interactive quiz application using Python.
- Use core Python programming constructs in a practical project.
- Store and manage quiz questions and their correct answers.
- Accept user input and validate the selected option.
- Check answers automatically and maintain a running score.
- Display a clear final result after the quiz is completed.
- Provide documentation that allows a new user to set up and run the project.

4. SCOPE OF THE PROJECT

The current scope covers a terminal-based quiz with multiple-choice questions. The basic application does not require a database, web server, or third-party Python package. Questions can be stored directly in the Python source code or in another simple data source if the implementation is extended.

The project is intended to demonstrate programming fundamentals rather than provide a full commercial quiz platform. Advanced functionality can be added as future enhancements.

5. TECHNOLOGIES AND REQUIREMENTS

Component	Requirement
Programming Language	Python 3.8 or later
Interface	Command Line / Terminal
Libraries	Python Standard Library
Operating System	Windows, macOS, or Linux
Development Environment	Any Python-compatible editor or IDE

6. SYSTEM DESIGN AND WORKFLOW

The quiz follows a straightforward sequence:

- Start the application.
- Display a welcome message and initialize the score.
- Select or read the next question.
- Display the question and its answer choices.
- Read the user's answer.
- Validate the input format.
- Compare the selected answer with the correct answer.
- Update the score when the answer is correct.
- Continue until all questions have been processed.
- Display the final score and completion message.

7. DATA STRUCTURE

A simple list of dictionaries can be used to store the question bank. Each dictionary contains the question text, the available options, and the correct answer.

```
questions = [
{
"question": "What is the output of 2 + 2?",
"options": ["3", "4", "5", "6"],
"answer": "4"
}
]
```

8. IMPLEMENTATION

8.1 Initialization

The program initializes the question collection and a score variable. The score starts at zero and is updated as the user answers questions.

8.2 Question Display

Each question is displayed with its available options. A loop can be used to process the questions one by one, avoiding repeated code.

8.3 Input and Validation

The program reads the user's response from standard input. Input validation can ensure that only permitted choices, such as A, B, C, or D, are accepted.

8.4 Answer Checking

After receiving a valid response, the program compares it with the stored correct answer. A correct response increases the score, while an incorrect response leaves the score unchanged.

8.5 Final Result

After all questions are answered, the application reports the user's score, for example 4 out of 5 questions answered correctly.

9. PROJECT STRUCTURE

A typical repository can use the following structure:

```
python-quiz-game/
■■■ README.md
■■■ quiz_game.py
■■■ requirements.txt
```

The requirements.txt file is optional for the standard-library version. README.md should remain at the root of the repository and provide setup, dependency, configuration, and execution instructions.

10. SETUP AND EXECUTION

Step 1: Install Python

Install Python 3.8 or later and verify the installation:

```
python --version
```

If necessary, use:

```
python3 --version
```

Step 2: Obtain the Repository

Clone the project repository and enter its directory:

```
git clone <YOUR_REPOSITORY_URL>
cd <PROJECT_DIRECTORY>
```

Step 3: Install Dependencies

The standard version requires no third-party packages. If a requirements.txt file is present, install it using:

```
pip install -r requirements.txt
```

Step 4: Run the Application

```
python quiz_game.py
```

Or, where required:

```
python3 quiz_game.py
```

11. SAMPLE EXECUTION

A typical command-line session may appear as follows:

```
Welcome to the Python Quiz Game!

Question 1: What keyword is used to define a function in Python?
A. function
B. def
C. func
D. define

Enter your answer: B
Correct!

Quiz completed!
Your score: 4/5
```

12. TESTING AND VALIDATION

Testing should verify both normal operation and invalid-input handling. The following test cases are recommended for the project.

Test Case	Input / Condition	Expected Result
Valid correct answer	A valid correct option	Score increases by one
Valid incorrect answer	A valid incorrect option	Score remains unchanged
Invalid option	An unsupported option	User is prompted again or input is rejected
All questions completed	Last question answered	Final score is displayed
Empty input	Press Enter without an answer	Input is handled without crashing

13. EXPECTED OUTCOME

When the application is correctly configured and executed, the user should be able to complete the quiz from the terminal, receive immediate feedback or answer validation according to the implementation, and see a final score at the end. The documented setup should allow an evaluator with no prior project context to install Python, obtain the repository, install any listed dependencies, and execute the application.

14. LIMITATIONS

- The basic version is terminal-based and does not provide a graphical interface.
- Questions are typically maintained in source code unless an external data file is added.
- The standard version does not provide persistent user accounts or stored results.
- Advanced features such as timers and leaderboards are outside the basic scope.

15. FUTURE ENHANCEMENTS

- Add difficulty levels such as Easy, Medium, and Hard.
- Add categories for different quiz subjects.

- Randomize question and answer order.
- Add a countdown timer.
- Store high scores in a file or database.
- Add a graphical user interface.
- Add a larger external question bank using JSON or a database.
- Add a restart option and session statistics.

16. CONCLUSION

The Python Quiz Game is a compact project that demonstrates how core Python programming features can be combined to create a useful interactive application. It covers input, processing, decision-making, iteration, data storage, validation, and output. The project also provides a clear foundation for adding more advanced features. With the accompanying README and structured documentation, the repository can be understood and executed by an evaluator without requiring prior knowledge of the project.

17. REFERENCES / RESOURCES

- Python documentation and standard-library concepts.
- Course/project instructions and repository requirements.
- Project source code and README documentation.

18. QUICK START

For a fresh setup, the essential workflow is:

```
git clone <YOUR_REPOSITORY_URL>
cd <PROJECT_DIRECTORY>
python quiz_game.py
```

End of Project Report

19. PROGRAM SOURCE CODE

Complete Python source code used for the Python Quiz Game

```
print("Welcome to the Python Quiz Game!")
print("------------------------------")

score = 0

# Question 1
answer = input("1. What keyword is used to create a function in Python? ")

if answer.lower() == "def":
    print("Correct!")
    score += 1
else:
    print("Wrong! The answer is def.")

# Question 2
answer = input("2. Which function is used to display text on the screen? ")

if answer.lower() == "print":
    print("Correct!")
    score += 1
else:
    print("Wrong! The answer is print.")

# Question 3
answer = input("3. Which symbol is used for comments in Python? ")

if answer == "#":
    print("Correct!")
    score += 1
else:
    print("Wrong! The answer is #.")

# Question 4
answer = input("4. What data type is 'Hello'?\n"
               "A. int\n"
               "B. str\n"
               "C. float\n"
               "D. bool\n"
               "Your answer: ")

if answer.lower() == "b":
    print("Correct!")
    score += 1
else:
    print("Wrong! The answer is B. str.")

# Question 5
answer = input("5. Which function gets input from the user? ")

if answer.lower() == "input":
    print("Correct!")
    score += 1
else:
    print("Wrong! The answer is input.")

print("------------------------------")
print("Quiz finished!")
print("Your score is:", score, "/ 5")

if score == 5:
    print("Excellent!")
elif score >= 3:
    print("Good job!")
else:
    print("Keep practicing!")
```